From Mined Fixes to Gated Repair: Closed-Loop Defect Synthesis for CIRCT

Abhinav Venkata Kota, Adithya Jillellamudi, Priyesh Shukla
International Institute of Information Technology, Hyderabad
{abhinav.kota, adithya.jillellamudi}@students.iiit.ac.in, priyesh.shukla@iiit.ac.in

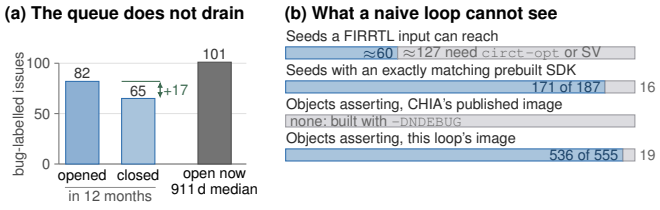


Fig. 1. Motivation. (a) CIRCT closes fewer bug issues than it receives, and what stays open is old. (b) What a mined-fix loop must overcome.

***Abstract*—A bug in a hardware compiler such as CIRCT can put a silent defect into a chip. Repair agents fix the bugs people report, but people report few. We built a closed loop on the CHIA agent framework that supplies its own bugs: a model reads each of 187 past CIRCT fixes, states the mistake behind it, and writes inputs that probe for the same mistake elsewhere. Deterministic tools build, judge, reduce and deduplicate every input, CHIA's repair chain drafts the patch, and a named person approves every filing. We also restored the assertions CHIA's published image had compiled out, in 536 of its 555 objects. In two campaigns the loop found fifteen new bugs, each reproduced, minimised, validated and unmatched against 2,652 issues, while a mutation baseline over 2,308 inputs found none. Both campaigns read 205 million tokens, 62% from cache, and wrote 3.7 million.**
Code and data:
github.com/DevAbhinav-23/CHIA_Hackathon_2k26

***Index Terms*—Hardware compilers, Program repair, Test generation, Compiler fuzzing, CIRCT.**

I. Introduction

CIRCT is an open-source compiler for hardware, built on MLIR [1]. A designer describes a circuit in FIRRTL or SystemVerilog, CIRCT lowers it to Verilog, and synthesis tools turn the Verilog into a chip. A crash is visible; wrong Verilog passes every tool below the compiler and can reach fabrication. CIRCT's regression tests run under LLVM's `lit`.

CHIA [2] is the agent framework this hackathon builds on. Its CIRCT issue solver takes a GitHub issue through six phases: assess, reproduce, fix, run the test suite, repair regressions, and write the pull request. On 16 CIRCT issues it fixed five, and maintainers merged three. Its bottleneck is supply. Over the past year CIRCT received 82 bug-labelled issues and closed 65; 101 stay open, at a median age of 911 days (Fig. 1a). A fix that corrects one wrong assumption inside a pass usually leaves the same assumption at other sites. CIRCT ships no fuzzer to find them, and a naive loop would miss most of them (Fig. 1b).

An agent reads each past fix and writes inputs aimed at those sibling sites, and tools discard almost everything it writes. Our contributions:

- A loop in which the model acts three times, reading a fix, writing inputs and writing the report, and a tool answers every gate (Section II, Fig. 2).
- Rebuilds of 171 of 187 historical CIRCT commits in minutes from prebuilt SDKs, and an image with assertions restored that passes the same tests (Section III, Table I).
- Thresholds pre-registered by git tag, a mutation baseline on the same harness, and a dedup rule measured on 22 labelled pairs (Section IV).
- Fifteen new CIRCT bugs through a four-question gate in two campaigns, against none from the baseline, every number read from the loop's database (Section V, Table II).

II. The Loop

Fig. 2 marks the stages the agent runs; tools decide everything else. One model serves every agent stage, Gemini 3.8 Flash through CHIA's Vertex backend, and no stage can reach it unless the operator sets one environment variable for that run, so tests and dry runs never call the model.

1) **Read the seed:** A seed is one of 187 CIRCT commits from the last two years that look like bug fixes, with its diff and the test its author added or changed. The agent states which root-cause class the fix corrected and names other sites where the same assumption still stands.
2) **Write probing inputs:** The seed's test command names the tool that reached the bug, `circt-opt`, `firtool` or the SystemVerilog front end, and so decides the language. About 127 of the 187 seeds sit in code no FIRRTL input can reach. The agent reads the compiler's source through its own tools and runs no command.
3) **Build:** CIRCT compiles against one pinned LLVM and MLIR commit (Section III), with CIRCT's assertions compiled in.
4) **Judge:** The primary oracle is unambiguous: the compiler crashed, or an assertion fired. A second oracle compares CIRCT's simulator with Verilator and only reports, because the two routinely disagree over uninitialised and reset signals.

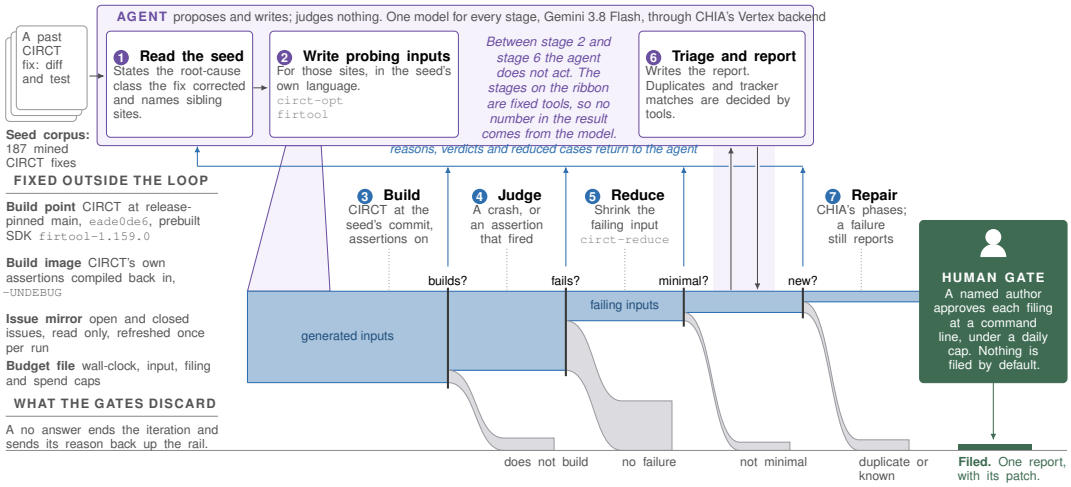


Fig. 2. Seeded CIRCT bug loop. The agent holds the violet band and acts three times only, at stages 1, 2 and 6. The blue ribbon below it is fixed apparatus, so no number in the result comes from the model. The ribbon is the candidates in flight, pinched at each gate by a yes-or-no question, and the share answering no peels onto the floor. Widths are illustrative.

TABLE I
WHAT WE MEASURED ON THE IMAGE THE LOOP RUNS

Measurement	Result
Building the past cheaply	
Fix-shaped commits mined from 24 months	187
Of those, seeds with an exactly matching prebuilt SDK	171
Pin windows with no matching release, worst 16 days	16 of 63
Letting the compiler complain	
Compile commands carrying `-UNDEBUG`	778 of 778
Object files carrying the assertion handler again	536 of 555
CHIA's whole test gate, assertions on and then off	1,119 pass, 0 fail, identical
What the assertions cost: wall clock, bytes, per probe	18%, 25%, 4%
Tests `lit` discovers, once its source root is the SDK	1,390, 0 errors

5) **Reduce:** `circt-reduce`, which CIRCT ships, deletes parts of a failing input while the failure survives; a textual reducer takes over where that tool dies.
6) **Triage and report:** A fingerprint is the assertion text with its source location or, for a crash, the symbolised frames below the crash handler. It matches survivors against a mirror of CIRCT's issues and against commits since the build point. The agent writes the report, and no gate reads what it concluded.
7) **Repair:** CHIA's phase chain runs unaltered except for one branch that selects our backend and an entry point that takes a local report in place of a GitHub issue. Repair is restricted to crashes and assertion failures.

The gates: Each gate is a yes-or-no question, a bar pinching the ribbon in Fig. 2; a no returns its reason to the agent with the verdicts and reduced cases. The last gate asks four questions, each answered by a tool: does the failure reproduce in a fresh process on another worker, is the case minimal, is the input valid, and is it new. Validity is three parse-and-verify commands; a clean rejection makes the input the loop's fault, not the compiler's. Then a named author approves every filing at a command line; by default the loop files nothing.

III. Measured Preconditions

The loop must build CIRCT at arbitrary points of its history, with assertions on. Neither held when we started; Table I lists what we measured on the image the loop now runs.

Building the past cheaply: CIRCT pins the exact LLVM commit it expects; a build against any other fails, and compiling that LLVM costs hours per seed. CIRCT also publishes prebuilt LLVM and MLIR SDKs carrying their own pin, and where the two pins match, CIRCT rebuilds in minutes: 171 of the 187 seeds do. The loop itself runs at release-pinned main, the newest commit whose pin has a published release, 8 commits and 4.8 days behind head for this image. A longer lag means more candidates come back already fixed.

Letting the compiler complain: CHIA's published CIRCT image defines `NDEBUG`, which deletes assertions, so an assertion oracle would find nothing under it. The flag `-UNDEBUG` puts them back; the 19 objects without the handler assert nothing. We built the image twice, differing in that flag alone, and ran CHIA's whole test gate in both: the same 1,119 tests pass and none fail, so a probe that trips an assertion trips it because of the probe. One defect needed repair first: at CIRCT commits after mid-2026 a test directory's configuration raises during `lit` discovery, CHIA's exclusion of that directory hid the error, and the gate discovered nothing. Using the SDK as the MLIR source root restores discovery.

Tested on real failures: We calibrated the oracle and fingerprints on six past CIRCT bugs, rebuilt at the commit before their fix. This exposed two fingerprinting defects: the name normaliser erased every frame in an anonymous namespace, where CIRCT's passes live, and the scope test read an inlined frame instead of the physical one. Together they put three of the four real bugs outside the repair stage's reach; we fixed both.

IV. Controls

The question is whether reading a past fix finds more new bugs than mutating the same tests. Two arms share the harness,

TABLE II
BOTH CAMPAIGNS, FROM THE LOOP'S DATABASE. A SEEDED COLUMN IS ITS TWO SHARDS TOGETHER. THE MUTATION COLUMN IS ONE RUN PER CAMPAIGN, AND THE TWO RUNS AGREE IN EVERY CELL

Measure	seeded 1	seeded 2	mutation
Seeds probed	9	43	96
Probes written	134	624	1,154
Output rejected by its own verifier	10	7	0
Assertion failures	5	54	0
Crashes	0	8	0
Candidates	15	69	0
Distinct new fingerprints	3	14	0
Through the gate	3	12	0
Filed	0	0	0
Model turns	57	191	0
Input tokens, millions (share from cache)	56.0 (68%)	148.7 (59%)	0
Output tokens, millions	0.72	3.00	0

oracles, reducer and gate; only the generator and its information differ. The mutation arm rewrites seed tests syntactically, on 96 of the 184 eligible seeds, without the diff or the root-cause class, after Mut4All's model-synthesised mutators [3]. Both arms get the same wall-clock window, but the mutation arm exhausts its mutants after 775 and 266 seconds; a post-review run that used the whole window gated 2 bugs from 115,869 inputs. The registered headline is distinct bugs a CIRCT maintainer confirms by labelling, commenting on or fixing them; until filing, we report bugs through the gate.

Pre-registration: Every threshold lives in one budget file, registered by an annotated git tag on the commit that lands it. The loop refuses an untagged file and a mutator set committed after the tag. Safety caps bound damage: 2,000 inputs per arm per day, 3 filings per day and 10 per run, 60 seconds and 4 GiB per probe, and a hard cap on model spending. Nothing is filed until a human has posted the method to the project's forum.

Deduplication: Two candidates are one bug when their primary fingerprints are equal. On 22 hand-labelled pairs the rule merges 1 of the 11 distinct pairs and splits 1 of the 11 duplicates.

Known issues: GitHub stops paginating at page 100, so the tracker mirror walks newest first, then oldest first, and unions the two: 2,652 issues in 198 requests.

Contamination: The model may have memorised later CIRCT fixes. We screen each candidate against fixes made after its seed, at the level of the symbols the seed's fix touched; this flags 77% of the corpus, against 96% at file level. A flag marks a candidate and does not exclude it.

V. RESULTS

Table II reports two registered campaigns, each running the seeded arm as two parallel shards. The first gave every arm two hours under a per-job spending cap; we stopped its first two starts early for defects the live run exposed. The second changed only three registered numbers: the whole corpus, a 41.7-hour window and a higher cap. We stopped it after 11.9 hours. Every number comes from the loop's database after re-judging stored probes under the corrections below, with no model turn re-run.

Stable yield: Fig. 3 reads Table II as rates. The seeded campaigns differ fivefold in size, yet yield 11.2 and 11.1 candidates and 2.2 and 1.9 gated bugs per 100 inputs. The mutation arm wrote three times as many inputs as both seeded campaigns together and none fired the oracle; only 34% reached the pass they target. A gated bug cost 18.7 million input tokens in the first campaign and 12.4 million in the second. Fifteen bugs establish a consistent rate, not an effect size.

Fifteen new bugs: The first campaign's three all come from the August `moore.extract` seed: a zero-element array trips the assertion at `HWOps.cpp:1995`, and a dynamic extract and an extract of a reference lower to `comb.extract` and `llhd.sig.extract` with aggregate types. The second campaign's twelve spread across the compiler; three are not assertions: a segmentation fault in `resolveClassStructBody` in `MooreToCore.cpp`, one whose only named frame is `main` in `circt-opt.cpp`, and an abort in `ElaboratedObjectOp::verifySymbolUses` in `OMOps.cpp`. All fifteen passed the four gate questions. They share twelve root causes. Re-run on CIRCT main of 26 October, built the same way, all fifteen still fail as the same bug, and no issue or commit since the pin reports or fixes any of them. Two more new fingerprints failed the gate: one did not reproduce, and the validity checker rejected the other.

The oracle had to learn a class: Seventeen rejected seeded probes were the compiler's fault, not the model's: the input alone parses and verifies, and the diagnostic has the shape of MLIR's generated verifier. We require both, since a pass may legitimately reject a valid input, such as one with two top-level modules. No mutation rejection qualified: a mutated test that fails to parse is broken syntax.

Four dedup corrections: Each was found on a real candidate. A post-pin commit counted as a fix if it touched the crash's file, so an LLVM bump touching 87 files silenced the first find; it now has to touch a frame symbol. A generic MLIR assertion text matched an open issue, and now also needs a CIRCT frame token. A verifier failure matched an issue on the op name alone, and now needs the constraint. Two candidates sharing a fingerprint were each the other's duplicate, which left thirteen without a representative. Verdicts are recomputed from stored evidence, so a corrected rule re-judges every candidate without a model turn.

Tokens: The campaigns read about 6.2 and 3.5 million input tokens per seed. Most of that is re-reading: CHIA's backend resends the whole conversation on every tool call, so a turn with twelve calls reads its context twelve times.

Repair: Of three attempts on the assertion, the first two never reached a patch because the loop's script handed the tool two input files. The third reproduced the failure, wrote a one-line change, rebuilt CIRCT, ran its test gate, and hit the 45-minute wall while writing up. The change was a debug print; we claim no fix from the chain. After the campaigns we wrote 13 patches with regression tests outside the loop, against that main: one per root cause and one for a further crash found on the way. Every reproducer now lowers to valid IR or stops with a diagnostic, each new test fails without its patch, and CIRCT's 1,432 lit tests give the same results before and after.

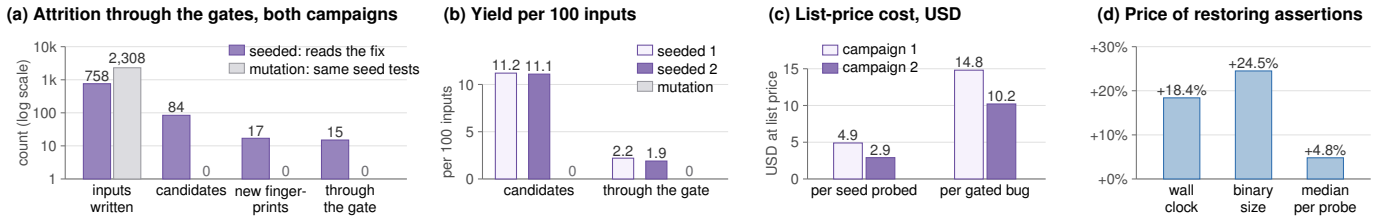


Fig. 3. Both registered campaigns (Table II). (a) One harness, oracle and gate; only the generator and its information differ. Zeros sit at the log axis floor. (b) Rates agree though campaign 2 is five times larger. (c) Model use per gated bug at list price, USD 0.75 and 3.75 per million input and output tokens. (d) Overheads of Table I.

None is submitted yet.

VI. Related Work

SWE-bench [4] scores agents on GitHub issues, and CHIA's issue solver [2] applies the pattern to a hardware compiler. Both draw from a queue, and we fill one. Fuzzers fill queues without repairing or withholding: CSmith [5] for C, and for MLIR, MLIRod [6], Nüwa [7], DESIL [8] and FLEX [9]. Our mutation arm follows Mut4All's synthesised mutators [3]. IssueMut [10] mines mutators from bug histories, and Fuzz4All [11] generates inputs with a model. AFuzz [12] is the origin of our method, an agent that tests where else a reference bug's cause could live; we add the CIRCT build, validation and triage around it, behind a gate. Delta debugging [13] is stage 5's algorithm. Abstain-and-Validate [14], llvm-autofix [15] and CodeRover-S [16] make a repair agent more selective. Our gate sits downstream and discards by default.

VII. Artefact and Limitations

The loop ships as CHIA nodes and tools meeting at a frozen contract of seven record types, with 739 tests, 689 needing no cluster, network or model. Upstream pull requests to CHIA carry the assertions-on image and its build workflow (ucb-bar/chia#117) and three helpers for its CIRCT module (#118); the repair-backend branch is not yet submitted. The corpus and every input, verdict, reduced case, transcript and gate decision are at github.com/DevAbhinav-23/CHIA_Hackathon_2k26.

Limitations: No maintainer has confirmed the bugs, and none is filed. The campaigns reached 43 of the 187 seeds, about 3.6 an hour across two shards under Vertex rate limits, so the list is not exhaustive. The second oracle applies to 6 of the 187 seeds, and the seeds rest on a subject-keyword filter: we read no diff to confirm that each is a bug fix. The contamination screen misses a sibling fixed in a commit whose subject does not name it. The loop finds crashes and assertion failures only; the wrong-output oracle only reports.

Acknowledgements and disclosure: We thank the CHIA developers and the A^3 organisers. Claude (Anthropic) helped draft text and code. The authors answer for every claim, number and decision.

VIII. Conclusion

Reading past fixes exposes sibling defects, restored assertions make them observable, and a gate of tools discards the rest. Fifteen new CIRCT bugs went through that gate where a mutation baseline sent none in the same campaigns, and a person decides whether each is filed.

References

[1] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. IEEE/ACM Int. Symp. on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[2] A. Cui *et al.*, "CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research," *arXiv preprint arXiv:2606.27350*, 2026.

[3] B. Wang *et al.*, "Mut4All: Fuzzing compilers via LLM-synthesized mutators learned from bug reports," *arXiv preprint arXiv:2507.19275*, 2025.

[4] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

[5] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2011, pp. 283–294.

[6] C. Suo, J. Chen, S. Liu, J. Jiang, Y. Zhao, and J. Wang, "Fuzzing MLIR compiler infrastructure via operation dependency analysis," in *Proc. 33rd ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2024.

[7] B. Cao, W. Tong, Z. Tang, Z. Wang, H. Huang, and Y. Yan, "Nüwa: Enhancing MLIR fuzzing with LLM-driven generation and adaptive mutation," in *Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME)*, 2025.

[8] C. Suo, J. Wang, Y. Wang, J. Jiang, Q. Shen, and J. Chen, "DESIL: Detecting silent bugs in MLIR compiler infrastructure," *arXiv preprint arXiv:2504.01379*, 2025.

[9] Z. Sun, J. Liang, W. Wang, C. Suo, J. Chen, and F. Xu, "Interleaved learning and exploration: A self-adaptive fuzz testing framework for MLIR," in *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2025.

[10] L. Liu, F. Qin, O. Legunsen, and M. d'Amorim, "Learning compiler fuzzing mutators from historical bugs," in *Proc. IEEE/ACM Int. Conf. on Mining Software Repositories (MSR)*, 2026.

[11] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2024.

[12] J. Park and I. Yun, "Agentic fuzzing: Opportunities and challenges," *arXiv preprint arXiv:2605.10074*, 2026.

[13] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.

[14] J. Cambronero *et al.*, "Abstain and validate: A dual-LLM policy for reducing noise in agentic program repair," in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, pp. 118–129.

[15] Y. Zheng, C. Li, S. Li, Y. Zhang, and Z. Su, "Agentic harness for real-world compilers," *arXiv preprint arXiv:2603.20075*, 2026.

[16] Y. Zhang, J. Wang, D. Berzin, M. Mirchev, and A. Roychoudhury, "Fixing security vulnerabilities with agentic AI in OSS-Fuzz," in *Proc. IEEE/ACM Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026.